Projet OCR : Résolution de Mots Croisés / Cachés

Rapport d'Avancement – Soutenance 1

Cahier des charges, architecture système et réalisations initiales

---

Équipe / Groupe :

Équipe : PA2_G1

Classe : PA2 – Promotion 2030

Membres et Rôles :

Responsable Traitement d'Images : Anh-Khoa Nguyen

Responsable Réseau de Neurones / OCR : Gabriel Croci

Responsable Solver : Mathéo Lechalupe

Responsable Interface / Intégration : Dorian Romagnoni

Date de soumission : Octobre 2026

Table des matières

1 Rôle et Mission de Chaque Membre **2**

2 Objectifs, Planification et État d'Avancement **3**

2.1 Objectifs et Planification du Solver 3

2.2 Objectifs et Planification du Réseau de Neurones (OCR) 3

2.3 Objectifs et Planification du Traitement d'Images 4

2.4 Objectifs et Planification de l'Interface 4

3 Choix Techniques, Justifications et Difficultés Rencontrées **5**

3.1 Module Traitement d'Image 5

3.1.1 Introduction et Apprentissage de la SDL2 5

3.1.2 Accès Mémoire et Gestion des Pixels 5

3.1.3 Chargement et Conversion en Nuances de Gris 5

3.1.4 Binarisation Adaptative par Algorithme d'Otsu 5

3.1.5 Nettoyage des Parasites Visuels (`remove_isolated`) 6

3.1.6 Validation et Exécution des Tests (`test_binarization.c`) 6

3.2 Module Reconnaissance Optique de Caractères (Réseau de Neurones) 7

3.2.1 Architecture du MLP 7

3.2.2 Entraînement du MLP 7

3.2.3 Validation de l'apprentissage du XNOR par un MLP 8

3.2.4 Début de l'apprentissage de reconnaissance de lettres 9

3.2.5 Sauvegarde et chargement des modèles 9

3.2.6 Utilisation du modèle 9

3.3 Module Résolution de la Grille (Solver) 10

3.3.1 Objectif 1 – Charger la grille 10

3.3.2 Objectif 2 – Rechercher un mot dans les huit directions 10

3.3.3 Objectif 3 – Erreurs et mémoire 11

3.3.4 Objectif 4 – Tests 11

3.4 Module Interface Utilisateur et Intégration 11

4 Bibliographie et Références **12**

1 Rôle et Mission de Chaque Membre

Traitement d'Images (Anh-Khoa Nguyen)

Le module de traitement d'images a pour mission de transformer l'image brute d'une grille de mots cachés (souvent issue d'une photo de smartphone) en une image lisible, bien contrastée et exploitable. Mon rôle couvre l'intégralité de la chaîne de prétraitement visuel : la prise en main de la bibliothèque SDL2, l'accès direct et sécurisé à la mémoire des pixels, la conversion en niveaux de gris avec gestion de la transparence alpha, ainsi que la binarisation adaptative par l'algorithme d'Otsu pour s'affranchir des variations d'éclairage et d'ombres. Enfin, je prends en charge le nettoyage des bruits isolés et la création de scripts de tests unitaires dédiés (`test_binarization.c`) afin de garantir la qualité de l'image transmise pour la suite du projet.

Reconnaissance Optique de Caractères (Gabriel Croci)

Ce module vise à la reconnaissance de lettres dans un mots mêlés sous le format d'une image. Chaque case contenant une lettre sera extraite et analysée par le module et traduite en caractère ASCII pour le module du solver. L'objectif est de créer un réseau neuronal sous la forme d'un MLP (Perceptron Multicouche) qui apprendra à reconnaître des lettres avec des motifs différents.

Pour ce faire, nous avons commencé par créer un réseau simple capable d'apprendre la porte logique XNOR. Ce réseau sera ensuite une base simple pour l'étendre à l'apprentissage de reconnaissance de lettres.

Solver (Mathéo Lechalupe)

Le Solver consiste à réaliser en C, un programme permettant d'identifier les mots à travers une grille de lettres dans un fichier texte. Le programme affiche la position de la première et de la dernière lettre du mot dans la grille (Ex : (x0,y0)(x1,y1)).

Notre mission couvre quatre points : lire et valider le fichier de grille, représenter la grille en mémoire, rechercher le ou les mots, et restituer le résultat dans le format demandé.

Le solver est conçu pour être le plus indépendant possible. Il n'entre aucunement ou presque en contact avec les autres programmes. Il reçoit un fichier texte, et applique la recherche directement.

Interface Graphique et Intégration (Dorian Romagnoni)

<à écrire> *(Présenter le rôle et la mission sur l'interface et l'intégration)*

2 Objectifs, Planification et État d'Avancement

2.1 Objectifs et Planification du Solver

Pour réaliser le solver, nous avons découpé en 4 parties les différentes étapes :

1. Charger la grille depuis un fichier texte.
2. Rechercher un mot dans les huit directions possibles et retrouver ses coordonnées.
3. Gérer les erreurs et la mémoire.
4. Tester le programme sur les exemples du sujet en format texte.

Pour la planification :

Tâche	**Prévu**	**Réel**	**Écart**
Lecture de la grille (`grid_load`)	04/10/2026	04/10/2026	0
Recherche dans les 8 directions (solver)	11/10/2026	11/10/2026	0
Programme principal et affichage (main) + Gestion des erreurs et tests	18/10/2026	18/10/2026	0

Avancement : Le solver est fonctionnel pour la recherche d'un mot à la fois, à l'horizontal, en verticale et en diagonale. Si un mot demandé n'est pas trouvé, on renvoie « Not found ». Il gère les cas en majuscules, tout comme ceux en minuscules. Il ne nous restera juste à décider si nous devons faire une recherche globale pour chaque mot au lieu d'appeler le solver pour chaque mot.

2.2 Objectifs et Planification du Réseau de Neurones (OCR)

Pour réaliser le module OCR, nous avons découpé le travail en 4 étapes principales :

1. Implémentation des structures du réseau, de la passe avant (*forwarding*) et de la sigmoïde.
2. Rétropropagation du gradient, calcul des dérivées partielles et descente de gradient avec gestion du taux d'apprentissage.
3. Résolution du problème non-linéaire du XNOR, correction du biais cyclique par mélange aléatoire et sauvegarde/chargement des modèles.
4. Évolution vers l'architecture $784 \to 128 \to 26$ pour la reconnaissance de lettres.

Pour la planification :

Tâche	**Prévu**	**Réel**	**Écart**
Structures, allocation et passe avant (`forward`)	04/10/2026	04/10/2026	0
Rétropropagation et descente de gradient (`train`)	11/10/2026	09/10/2026	-2j
Apprentissage XNOR, shuffle et sérialisation (`save/load`)	18/10/2026	16/10/2026	-2j
Préparation de la nouvelle architecture ($784 \to 128 \to 26$)	25/10/2026	22/10/2026	-3j

Avancement : Le réseau de neurones est pleinement fonctionnel et validé sur la porte logique non-linéaire XNOR avec une convergence stable des poids et des biais. Le module de sauvegarde et de rechargement des modèles est opérationnel, et la fonction `test_results` permet de vérifier directement les prédictions. Nous sommes en avance sur le calendrier prévu : la structure à 784 entrées et 26 sorties est déjà instanciée en C, prête pour l'apprentissage futur des caractères.

2.3 Objectifs et Planification du Traitement d'Images

Pour le traitement d'image avec la SDL2, nous avons structuré le pipeline en 4 étapes :

1. Prise en main de la SDL2 et accès mémoire direct aux pixels via `surface->pitch`.
2. Chargement des images et conversion en niveaux de gris avec intégration de la luminance et du canal alpha.
3. Binarisation adaptative automatique par l'algorithme d'Otsu.
4. Filtrage du bruit par détection des pixels isolés (`remove_isolated`) et validation dans `test_binarization.c`.

Pour la planification :

Tâche	**Prévu**	**Réel**	**Écart**
Apprentissage SDL2 et fonction `get_pixel`	04/10/2026	04/10/2026	0
Conversion en niveaux de gris (`load_grayscale`)	11/10/2026	11/10/2026	0
Binarisation adaptative (`otsu_threshold`)	18/10/2026	17/10/2026	-1j
Filtrage des parasites (`remove_isolated`) et tests	25/10/2026	23/10/2026	-2j

Avancement : La chaîne complète de prétraitement d'image est entièrement réalisée et validée. Le programme de test produit avec succès les fichiers intermédiaires (`out_gray.png`, `out_binaire.png`, `out_cleaned.png`). Les images binarisées et déparasitées sont prêtes à être transmises aux algorithmes de détection de la grille (transformée de Hough) pour la Soutenance 2.

2.4 Objectifs et Planification de l'Interface

<à écrire> *(Tableau de planification et avancement spécifique à l'Interface)*

3 Choix Techniques, Justifications et Difficultés Rencontrées

3.1 Module Traitement d'Image

3.1.1 Introduction et Apprentissage de la SDL2

Au début du projet, nous ne savions pas trop par où commencer ni comment aborder la partie traitement d'image d'une grille de mots cachés, ce qui constitue une erreur classique quand on débute. Heureusement, c'est grâce au TP 3 de l'école sur la SDL2 que tout s'est éclairci : cela nous a grandement facilité la compréhension du fonctionnement global d'un écran, de la manipulation des pixels et de leurs valeurs de couleurs.

Nous avons ainsi compris qu'il fallait explorer l'image méthodiquement en utilisant des boucles imbriquées pour parcourir chaque ligne et chaque colonne en fonction de la hauteur (`surface->h`) et de la largeur (`surface->w`).

3.1.2 Accès Mémoire et Gestion des Pixels

Au tout début, nous avions commis l'erreur d'essayer d'analyser les pixels sans faire attention aux limites, ce qui provoquait des bugs de mémoire. Pour régler ce problème proprement, nous avons mis en place une fonction `get_pixel` qui calcule la position exacte de chaque pixel en mémoire grâce à l'expression :

$$\text{Adresse} = (\text{Uint8}*)\text{surface} \rightarrow \text{pixels} + y \cdot \text{surface} \rightarrow \text{pitch} + x \cdot 4$$

Cela nous a permis d'accéder directement aux 4 octets RGBA de chaque pixel sans difficulté d'alignement mémoire.

3.1.3 Chargement et Conversion en Nuances de Gris

La première étape concrète de notre travail a ensuite consisté à charger l'image avec la fonction `load_grayscale` pour la transformer en nuances de gris. Nous avons dû ajuster le calcul mathématique des couleurs en combinant la luminance relative avec la formule usuelle :

$$\text{Luminance} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

De plus, nous avons pensé à intégrer la gestion de la transparence avec le canal alpha pour éviter d'avoir de vilains fonds noirs inattendus :

$$\text{Gris}_{\text{final}} = \text{gray} \cdot \alpha + 255.0 \cdot (1.0 - \alpha)$$

3.1.4 Binarisation Adaptative par Algorithme d'Otsu

Ensuite, pour préparer la binarisation en noir et blanc, nous avions failli utiliser un seuil fixe arbitraire à 128, mais nous nous sommes vite rendu compte que c'était une mauvaise idée : si une photo prise avec un smartphone présente des ombres, toute l'image risque de ressortir intégralement noire ou blanche et on perd tout le texte de la grille.

Pour corriger cela de façon intelligente, nous avons intégré la fonction `otsu_threshold` :

1. Elle construit un tableau d'histogramme de 256 valeurs.
2. Elle calcule la variance inter-classe (`varBetween`) pour tester chaque seuil possible.

3. Elle détermine automatiquement le meilleur seuil optimal.

Une fois le seuil trouvé, on l'applique avec la fonction `apply_threshold` via une condition ternaire :

```
p[0] = (p[0] < threshold) ? 0 : 255;
```

Ceci permet de basculer chaque pixel soit en noir absolu (0), soit en blanc pur (255).

3.1.5 Nettoyage des Parasites Visuels (`remove_isolated`)

Pour aller plus loin et rendre l'image parfaitement propre, nous avons créé la fonction `remove_isolated` qui élimine les petites poussières et le bruit visuel :

- Sécurité mémoire : Le parcours évite les bordures strictes de l'image en démarrant à $x = 1$ et $y = 1$.
- Analyse de voisinage : Pour chaque pixel noir, on vérifie ses 8 voisins directs.
- Filtrage : Si les 8 voisins sont blancs (`white_neighbors == 8`), le pixel est identifié comme un parasite isolé et transformé en pixel blanc.

3.1.6 Validation et Exécution des Tests (`test_binarization.c`)

Enfin, pour tester et valider tout ce travail étape par étape sans mauvaise surprise, nous avons conçu le programme dédié `test_binarization.c`. Il nous permet de tout orchestrer depuis le terminal en vérifiant les arguments d'entrée (`argc < 2`), puis de sauvegarder des images intermédiaires pour validation visuelle :

- `out_gray.png` : Image convertie en niveaux de gris avec gestion du canal alpha.
- `out_binaire.png` : Image binarisée selon le seuil calculé par Otsu.
- `out_cleaned.png` : Image finale filtrée de ses bruits isolés.

Cette chaîne garantit la transmission d'une image d'excellente qualité au reste de l'équipe pour les étapes ultérieures d'extraction de la grille et d'OCR.

3.2 Module Reconnaissance Optique de Caractères (Réseau de Neurones)

3.2.1 Architecture du MLP

Description d'un MLP : Un Perceptron Multicouche est une architecture de réseau neuronal simple qui permet de résoudre des problèmes non-linéairement séparables. Il est constitué de nœuds d'entrée (*inputs*), de nœuds cachés (*hidden nodes*) qui sont des détecteurs de motifs, et enfin de nœuds de sorties (*outputs*). Chaque nœud a un biais et une « valeur », qui représente son niveau d'activation entre 0 et 1. Tous les inputs sont liés aux hidden nodes, eux-mêmes tous liés aux outputs par des arêtes pondérées.

Le niveau d'excitation d'un nœud j est représenté par la somme pondérée de ses entrées augmentée de son biais :

$$z_j = \sum_i w_{ij} x_i + b_j \tag{1}$$

où w_{ij} désigne le poids reliant l'entrée x_i au nœud j, et b_j est le biais associé à ce nœud.

L'activation d'un nœud est ensuite donnée par l'application de la fonction sigmoïde :

$$a_j = \sigma(z_j) \tag{2}$$

Le problème de non linéarité : On utilise la sigmoïde pour avoir l'activation d'un nœud car combiner des multiplications et des additions produit une fonction affine. Or une droite ne peut pas séparer les 0 des 1. Ainsi la sigmoïde introduite par la formule suivante :

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{3}$$

permet de compresser n'importe quelle valeur dans l'intervalle $]0, 1[$, ce qui permet d'interpréter la sortie comme une probabilité ou un état binaire dans le cas de la porte XNOR. Sa dérivée s'exprime très simplement à partir de sa propre valeur : $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

3.2.2 Entraînement du MLP

La passe avant (*Forwarding*) : Au début de l'entraînement, tous les biais et poids du réseau sont initialisés de manière aléatoire. La passe avant consiste à faire tourner le réseau des inputs vers les outputs. On calcule l'ensemble des activations des nœuds pour obtenir à la fin des sorties qui seront très certainement incohérentes. L'objectif de la première passe est d'avoir un premier regard des résultats même s'ils sont faux. Cette étape reste nécessaire car elle servira à l'étape suivante qui détectera les erreurs.

La passe arrière ou rétropropagation (*Backpropagation*) : En sortie, on mesure l'écart entre la prédiction donnée $\hat{y}_k$ et la cible attendue y_k via l'erreur quadratique :

$$E = \frac{1}{2} \sum_k (\hat{y}_k - y_k)^2 \tag{4}$$

La rétropropagation part cette fois de l'erreur en sortie et remonte couche par couche en calculant la responsabilité exacte de chaque poids, soit pour un poids w donné :

$$\frac{\partial E}{\partial w} \tag{5}$$

— Si $\dfrac{\partial E}{\partial w} > 0$, il faut diminuer le poids car l'augmenter ferait grimper l'erreur.
— Si $\dfrac{\partial E}{\partial w} < 0$, il faut augmenter le poids.

La descente de Gradient : Pour le moment, aucun changement sur le neurone n'a été effectué : il faut désormais mettre à jour les poids pour prendre en compte les ajustements de la rétropropagation. La descente de Gradient s'occupe justement d'appliquer ces corrections : soit w un poids et η (ou lr) le taux d'apprentissage :

$$w \leftarrow w - \eta \cdot \frac{\partial E}{\partial w} \tag{6}$$

Le taux d'apprentissage (*learning rate*) représente la taille des pas que fait le réseau pour corriger ses poids vers le minimum d'erreur. Il doit être suffisamment bien dosé pour ne pas bloquer le réseau dans son apprentissage (trop grand = l'erreur oscille ou diverge, trop petit = le réseau bloque dans un minimum local).

Conclusion : L'apprentissage d'un MLP consiste à l'ajustement de ses poids, en fonction du résultat de ses sorties (prédictions) et ses cibles associées. Ce processus doit être répété un certain nombre d'époques avec un taux d'apprentissage bien gradué et une quantité suffisante d'entrées et de cibles.

3.2.3 Validation de l'apprentissage du XNOR par un MLP

Grâce à ces méthodes, on peut mettre en place un MLP simple de cette forme :

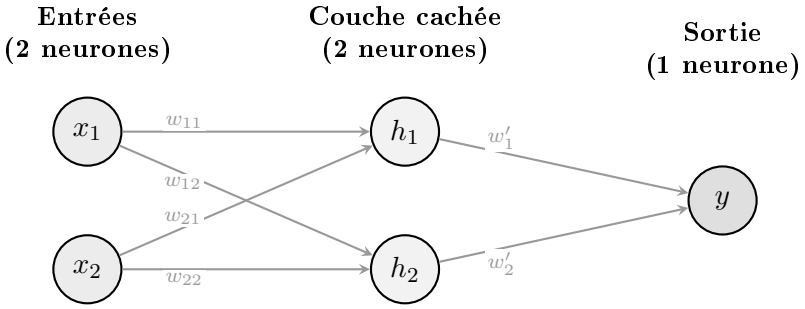


FIGURE 1 – Architecture du réseau de neurones pour l'apprentissage du XNOR (2-2-1) avec représentation des poids.

En C, on a représenté ce réseau dans un `struct` avec des matrices pour les poids et les biais de chaque nœud.

Pour l'apprentissage du XNOR, il nous a suffi de rentrer sa table de vérité en entrée, tout en alternant l'ordre. Nous avons remarqué que rentrer constamment les entrées dans le même ordre, par exemple :

$$(0,0) \longrightarrow (0,1) \longrightarrow (1,0) \longrightarrow (1,1)$$

donnait un biais cyclique au réseau. Il apprend la séquence plutôt que la logique en elle-même. Ainsi nous avons mélangé l'ordre des entrées à chaque époque d'entraînement lors de la descente de Gradient. Techniquement, c'est une descente de Gradient stochastique.

C'est un bon réflexe que nous avons adopté pour le futur entraînement de la reconnaissance de lettres.

3.2.4 Début de l'apprentissage de reconnaissance de lettres

Intuitions et débuts d'implémentation : Pour rentrer dans le cœur de l'OCR, nous avons préparé la nouvelle architecture adaptée à la reconnaissance des caractères :

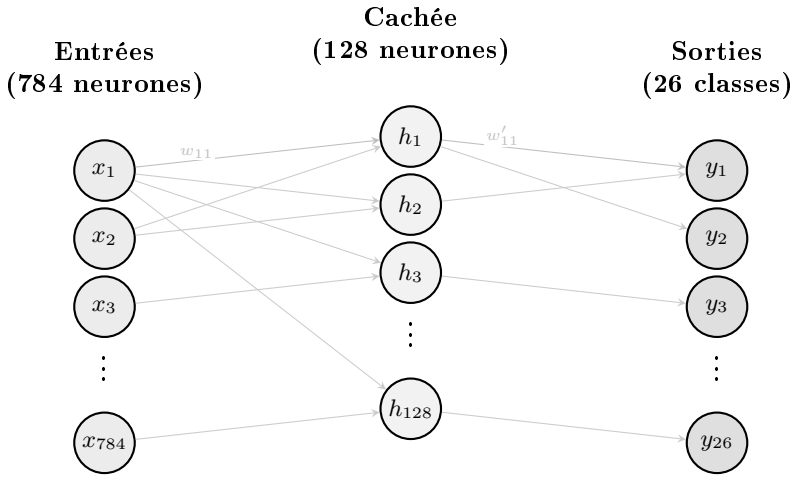


FIGURE 2 – Architecture étendue du MLP 784 → 128 → 26 pour la reconnaissance de lettres.

Cette fois il aura en entrée 784 nœuds correspondant chacun à un pixel du quadrillage où la lettre sera présente. Un pixel « allumé » (en noir) sera un 1 et un pixel « éteint » (donc blanc) sera un 0. Nous avons pris arbitrairement 128 nœuds cachés, ce nombre aura peut-être changé lors de la prochaine soutenance. Et enfin 26 sorties qui représentent chacune d'elle l'indice d'une lettre dans l'alphabet (0 = A, 1 = B etc). À la fin, la plus grande prédiction des 26 sorties sera prise en compte, et son indice correspondra à la lettre prédite par le MLP.

3.2.5 Sauvegarde et chargement des modèles

Ce module permet de sauvegarder des modèles après leur entraînement dans un fichier binaire. Il suit ce format :

```
# weight_hidden
// values

# bias_hidden
// values

# weight_output
// values

# bias_output
// values
```

On y sauvegarde les poids et biais de chaque nœud caché et de sortie. Pour le charger, au lieu d'initialiser un réseau avec des valeurs aléatoires, on lui associe cette fois le contenu du fichier.

3.2.6 Utilisation du modèle

Après avoir entraîné et sauvegardé un modèle, on peut l'utiliser en reprenant le concept de la passe avant (le Forwarding). On appelle ainsi la fonction `forward` qui prend en paramètres le

modèle et les entrées. Et enfin pour voir les résultats on utilise `test_results` qui affiche les prédictions après la passe avant.

3.3 Module Résolution de la Grille (Solver)

3.3.1 Objectif 1 – Charger la grille

La grille est stockée dans une structure `grid` contenant un tableau de lignes (`char **cells`), la largeur `w` et la hauteur `h`. Chaque ligne est un tableau de caractères alloué dynamiquement, à la manière d'une grille de lettres stockée dans un tableau à deux dimensions (voir [5]) et une lettre s'obtient par `cells[ligne][colonne]`. Ce choix s'explique par le fait que le fichier peut avoir n'importe quelle taille : ni la largeur ni la hauteur ne sont connues avant la lecture. Nous avons donc utilisé `getline` plutôt que `fgets`, car `getline` agrandit son tampon automatiquement et supprime toute limite sur la longueur d'une ligne (voir [1]). Pour la hauteur, nous agrandissons le tableau de lignes avec `realloc` au fur et à mesure. La première ligne fixe la largeur, et toute ligne de longueur différente fait échouer le chargement, ce qui garantit une grille rectangulaire. Les lignes vides et les caractères de fin de ligne (`\n`, `\r`) sont ignorés, et les lettres sont converties en majuscules dès la lecture afin que la recherche n'ait plus à se préoccuper de la casse.

Cette solution a toutefois posé plusieurs difficultés. D'abord, avec l'option `-std=c99`, `getline` n'est pas déclarée par défaut ; il faut définir `_POSIX_C_SOURCE` avant les inclusions. Ensuite, l'ordre des indices est un piège : `cells[y][x]` accède à la ligne avant la colonne, ce qui est facile à inverser. Enfin, il a fallu gérer les échecs d'allocation : le résultat de `realloc` est d'abord placé dans une variable temporaire pour ne pas perdre l'ancien pointeur, et la hauteur n'est incrémentée qu'une fois la ligne réellement ajoutée, afin que la libération de la mémoire ne parcoure jamais une case invalide.

3.3.2 Objectif 2 – Rechercher un mot dans les huit directions

L'algorithme parcourt toutes les cases de la grille. Lorsqu'une case contient la première lettre du mot, il teste les huit directions grâce à un tableau de déplacements (dx, dy), et la fonction `check_dir` répond à la question : le mot commence-t-il ici et part-il dans cette direction ? Ce tableau de directions remplace huit blocs de code quasi identiques par une seule boucle, ce qui réduit les risques d'erreur et facilite la lecture. Parcourir toutes les cases puis tester chaque direction est aussi la démarche proposée dans des sujets d'algorithmique universitaires (voir [3] et [4]). Cette approche est celle que l'on retrouve pour ce problème (voir [2]) : un couple de tableaux `dx` et `dy` pour les huit déplacements, et un arrêt dès la première lettre différente. Nous l'avons préférée à des algorithmes de recherche de motif plus élaborés comme Knuth-Morris-Pratt (voir [2]), qui impliqueraient d'extraire chaque ligne, colonne et diagonale, dans les deux sens. Pour des grilles de cette taille, la complexité au pire de l'ordre de $w \times h \times 8 \times n$ comparaisons (avec n la longueur du mot) est largement suffisante, et l'arrêt anticipé sur la première lettre la réduit fortement en pratique. La principale difficulté était de ne jamais lire en dehors de la grille. Pour cela, `check_dir` calcule avant toute comparaison la position de la dernière lettre et renvoie un échec si elle sort de la grille ; aucune case n'est donc lue hors limites (précaution également présentée dans [4]). Les coordonnées de fin sont ensuite calculées avec la même formule, ce qui évite les incohérences. Enfin, le sens de l'axe vertical, qui croît vers le bas puisque (0,0) est en haut à gauche, a demandé de la vigilance pour les directions « haut » et les diagonales.

3.3.3 Objectif 3 – Erreurs et mémoire

Nous avons géré chaque allocation de mémoire avec des fonctions auxiliaires permettant d'éviter des oublis de `free()` ou des dépassements. La difficulté vient de la multiplication des cas d'erreur (fichier absent, vide, lignes inégales, échec d'allocation).

3.3.4 Objectif 4 – Tests

Les tests couvrent les exemples du sujet, les grilles de niveau 1, et des cas limites : fichier absent, fichier vide, lignes de longueurs différentes, grille rectangulaire. Le programme est aussi compilé avec `-fsanitize=address` pour détecter les erreurs d'accès mémoire et les fuites. Tester séparément les directions et les cas limites est aussi la démarche d'autres solveurs publics (voir [6]).

3.4 Module Interface Utilisateur et Intégration

<à écrire> *(Présenter les choix techniques sur l'interface graphique et l'intégration globale)*

4 Bibliographie et Références

1. Page de manuel `getline(3)` (lecture d'une ligne avec tampon redimensionné automatiquement) : `https://www.mankier.com/3/getline`
2. Coding Ninjas (Naukri Code360), « Word Search - II » (parcours des huit directions avec des tableaux dx et dy , comparaison avec l'algorithme de Knuth-Morris-Pratt) : `https://www.naukri.com/code360/problem-details/word-search_630520`
3. Université de Washington, CSE 332, sujet d'exercice sur un solveur de mots mêlés (recherche dans les huit directions) : `https://courses.cs.washington.edu/courses/cse332/25su/exercises/specs/ex5spec.pdf`
4. Educative, « Hacker Challenge : Word Search Solver » (lecture de la grille, vérification des limites avant la recherche) : `https://www.educative.io/courses/beginner-to-advancedcomputing-a hacker-challenge-word-search-solver`
5. CS50, « Walkthrough : Problem Set 3 » (Scramble), vidéo YouTube, 1 h 02 (tableaux à deux dimensions pour représenter une grille de lettres, structuration en fonctions) : `https://rosetta.to/u/cs50/walkthrough-problem-set-3`
6. Man0may, « Word-Search-Solver », projet GitHub (solveur dans les huit directions testé cas par cas) : `https://github.com/Man0may/Word-Search-Solver`
7. N. Otsu, « A threshold selection method from gray-level histograms », *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
8. Simple DirectMedia Layer (SDL2) Documentation Wiki : `https://wiki.libsdl.org/SDL2/`
9. M. A. Nielsen, « Neural Networks and Deep Learning », Determination Press, 2015. Accessible en ligne : `http://neuralnetworksanddeeplearning.com/`
10. 3Blue1Brown (G. Sanderson), « Neural Networks », série vidéo pédagogique (chapitres 1 à 4 : architecture, descente de gradient, rétropropagation). Accessible en ligne : `https://www.3blue1brown.com/topics/neural-networks`
11. D. E. Rumelhart, G. E. Hinton, R. J. Williams, « Learning representations by back-propagating errors », *Nature*, vol. 323, pp. 533–536, 1986.